HCL Technology
Greater Noida

AI-Based Sales Conversion Probability Prediction



Session: 2025-26

Name: Shristi Sharma
Roll No : 2301330540043
Course: B. Tech
Branch: Biotechnology

IT Engineering Centre
HCL Technology

Plot Number-8, NSL TechZone, Sector 144,
Greater Noida - 201306, Uttar Pradesh

Table of Contents

1. Introduction
2. Data Understanding
3. Model Development
4. Model Evaluation
5. Results and Findings
6. Deployment
7. Challenges Faced
8. Future Enhancements
9. Conclusion
10. References

Team Members

Team Member	Roll No	Role	Responsibilities
Anshuman Singh	2301330100039	Data Scientist & Machine Learning Lead	Trains and optimizes the Random Forest classification engine; builds feature vectors and integrates Explainable AI (SHAP) diagnostics.
Shristi Sharma	2301330540043	UI/UX Developer	Designs the interactive Streamlit workspace; creates dynamic CSS layouts for custom runtime Light and Dark mode toggling.
Akshat Bansal	2301331520018	Data & Cloud Specialist	Manages data preprocessing and de-duplication; builds responsive Plotly visualization gauges; deploys app to cloud via GitHub.

Chapter 1 — Introduction

1.1 Problem Statement

Modern enterprise sales structures suffer from deep operational friction caused by unoptimized engagement cycles, static CRM pipeline management, and manual tracking systems prone to subjective human bias. This project develops an end-to-end, AI-driven Lead Conversion Prediction System that leverages machine learning engines to scoring prospective client conversion probabilities based on multi-dimensional behavioral interaction profiles. The solution addresses data processing, descriptive diagnostic reporting, and real-time inference automation to eliminate structural sales pipeline anomalies.

1.2 Business Context & Industry Focus

The project is situated within the highly competitive **Digital Business Architectures and Customer Relationship Management (CRM)** industry sector. Rapid conversion velocity and data-driven client profiling directly control market share expansion, pipeline scalability, and enterprise revenue performance.

1.3 The Real-World Problem (Describe the Pain Point)

Without this predictive solution, several critical vulnerabilities degrade enterprise sales workflows:

- **Static Pipeline Auditing:** Revenue teams review consumer engagement retroactively—often after a lead goes cold—resulting in permanently lost revenue opportunities.
- **Preventive Outreach Fatigue:** Marketing channels broadcast generic follow-up sequences at fixed, arbitrary intervals without analyzing consumer intent signals, creating brand fatigue and wasting human capital.
- **Inaccurate Revenue Forecasting:** Reliance on manual data entry and subjective sales notes introduces human bias, causing corrupted lifecycle documentation and unstable analytics.
- **Resource Allocation Mismatch:** High-intent buyers are left waiting due to processing latency, while extensive manual labor is exhausted on low-yield prospects.

1.4 Who You're Building For (Target Stakeholders)

- **Sales Managers & Administrators:** Need centralized dashboards to audit batch leads, allocate personnel based on priority bands, and evaluate core performance metrics.
- **Revenue/Sales Development Representatives (SDRs):** Require immediate, single-lead simulation tools to capture high-intent buyers before conversion windows close.
- **CRM Integration Architects:** Require automated data pipelines capable of ingesting webhooks and streaming predictive alerts to underlying enterprise stacks.

1.5 Why Your Solution Matters (Concrete Impact)

The deployment of this predictive framework yields quantifiable operational benefits:

- **Minimized Customer Acquisition Cost (CAC):** Eradicates overhead by systematically automating background nurture tracks for low-priority leads.
- **Enhanced Agent Closure Velocity:** Empowers enterprise reps to isolate and target high-yield leads instantly, reducing sales handling cycles.
- **Optimized Customer Lifetime Value (LTV):** Long-term pipeline health is stabilized by establishing automated, hyper-personalized workflow triggers tailored to consumer interactions.

1.6 Project Objectives

- **Business Objective:** Deliver a secure, real-time decision support system enabling revenue teams to route high-intent targets instantly to premium sales pipelines for immediate custom outreach within 15 minutes.
- **Analytical Objective:** Quantify the explicit correlation between user interaction distributions (e.g., website time spent, click-through rates, form completion counts) and eventual conversion success using Explainable AI (XAI) feature relevance models and SHAP tree structures.
- Machine Learning Objective: Construct a robust, serialized binary classification framework mapping the engineered lead feature vector to a target conversion outcome (0 for non-conversion, 1 for conversion). The target model performance relies on optimizing Recall, Precision, Accuracy, F1-Score, and ROC-AUC to prevent systemic model bias.

1.7 Scope of the Project

What is Included:

- Automated data preprocessing, tabular cleaning, categorical label mapping, and feature engineering for up to 30 customer tracking metrics.
- Training, hyperparameter constraint management, and serialization of an ensemble Random Forest Classification engine.
- Development of a dual-mode (Light/Dark) reactive visualization workspace application via Streamlit integrating real-time single simulation and batch prediction workflows.
- Post-prediction model interpretation via Explainable AI tracking mechanisms.

What is Excluded:

- Multi-class classification modeling for different specific product tiers or post-purchase subscription levels.

- Production-level database hosting setups (e.g., maintaining persistent external PostgreSQL or MongoDB servers within the Streamlit layer).
- Real-time automated voice processing or telephony analysis for live telemarketing call recordings.

Chapter 2: Data Understanding

2.1 Data Sources

The primary datasets containing multi-dimensional customer interaction records were obtained from open-source CRM repository logs modeled after benchmark data mining frameworks. The structured interaction matrix forms the basis for extracting customer behavioral patterns before manual sales routing takes place.

2.2 Dataset Description

The predictive framework ingests structured lead attributes such as lead source, industry, company size, budget, interaction count, response time, location, engagement score, and previous contact history. The preprocessing pipeline then creates engineered variables including Lead Value Score, Interaction Velocity, Engagement Per Interaction, and High Intent Lead before encoding and scaling the final model-ready feature matrix.

Column Name	Data Type	What it Represents / Description	Typical Range / Values
Age	Integer	Represents the chronological age of the prospective customer.	18 to 80
Time Spent on Website	Integer / Float	Evaluates the total absolute duration (in minutes) an active lead remains engaged on the web domain platform.	1 to 300 minutes
Pages Viewed	Integer	Quantifies the total count of individual URL page paths visualised by the customer during active sessions.	1 to 50 pages
Email Sent	Binary Integer	Historical interaction flag recording whether an outreach email template has been distributed to the target profile ($0 = \text{No}, $1 = \text{Yes}).	0, 1
Form Submissions	Integer	Tracks the total volume of specific inbound form completions and registration items executed by the customer.	0 to 20 submissions
Downloads	Integer	Computes the frequency of digital whitepapers, product resources, or documentation items downloaded by the lead.	0 to 20 downloads
CTR Product Page	Float / Percentage	Tracks the specific Click-Through Rate (100% scale) recorded on key product value-proposition pathways.	0% to 100%
Response Time	Integer	Meaures the operational handling latency (in hours) spent before a customer service representative responds.	0 to 72 hours

Column Name	Data Type	What it Represents / Description	Typical Range / Values
Follow-Up Emails	Integer	Records the absolute count of secondary nurturing touchpoint sequences transmitted to the contact.	0 to 20 emails
Social Media Engagement	Integer	Aggregate interaction score summarizing micro-conversions, comment signals, and brand impressions across platforms.	0 to 500 actions
Lead Source	Categorical String	Represents the inbound source channel pipeline where the contact first discovered the enterprise portal.	Email, Organic, Referral, Social Media
Payment History	Categorical String	Categorizes prior transactional and billing standing records associated with the user profile.	Good, No Payment

2.3 Initial Data Exploration

2.3.1 Missing Values and Fixing

The operational lead dataset undergoes automated data cleaning procedures to intercept data-entry errors. Missing fields or unrecorded tracking parameters within continuous behavioral streams are isolated and handled via algorithmic imputation techniques (such as replacing structural null vectors with statistical column medians), ensuring a normalized downstream processing pipeline.

2.3.2 Outliers and Fixing

Extreme values resulting from automated web crawlers or noisy bot traffic (e.g., abnormally inflated website time durations or hyper-frequent page hits) are isolated. Outlier bounds are checked using interquartile range rules to prevent extreme anomalies from shifting the splitting nodes of the underlying decision tree matrix.

2.3.3 Class Distribution

The dataset is audited for systemic class imbalance between converted (1) and non-converted (0) outcomes. To ensure the predictive framework remains sensitive toward locating high-value targets, balanced weighting techniques are integrated during the backend model-fitting process to counter any natural data skew.

2.3.4 Encoding

To prepare the dataset for backend matrix training operations, categorical marketing variables—specifically Lead Source and Payment History—are converted into numeric formats utilizing advanced vector mapping schemes. For example, categorical entries like *Organic*, *Referral*, and *Social Media* are expanded into multi-column binary metrics to yield a pure numerical matrix.

2.3.5 Feature Engineering

To maximize prediction accuracy, business-focused relational features are constructed from raw lead interactions: Lead Value Score, Interaction Velocity, Engagement Per Interaction, and High Intent Lead. These engineered variables help capture hidden relationships between budget, engagement, response speed, and outreach frequency that influence conversion outcomes.

Chapter 3: Model Development

3.1 Model Selection: Logistic Regression and Random Forest Comparison

For this project, two supervised binary classification models are implemented and compared: Logistic Regression and Random Forest. Logistic Regression provides a clear interpretable baseline, while Random Forest captures non-linear lead behavior. The final champion model is selected using evaluation metrics on a stratified holdout test set.

- Model Diversity: Logistic Regression gives a transparent baseline for probability estimation, while Random Forest captures non-linear customer journey patterns such as response time combined with engagement score.
- Generalization Control: The models are evaluated on a separate stratified test set, and Random Forest uses constrained tree depth to reduce overfitting to noisy lead behavior.
- Robustness to Mixed Feature Types: Numerical features are standardized and categorical attributes are one-hot encoded, allowing both models to process CRM-style tabular data consistently.
- Explainability Support: The dashboard combines model metrics with SHAP and LIME explanations, helping users understand which lead attributes increase or decrease conversion probability.

3.2 Model Training: Training Process and Pipeline Execution

The training architecture is structured as a programmatic pipeline implemented in Python utilizing the Scikit-learn machine learning engine. The execution flow proceeds as follows:

1. Matrix Ingestion and Splitting: The structured dataset is cleaned, encoded, scaled, and split into distinct training and testing subsets using an 80-20 stratified split. The training partition is used to fit both Logistic Regression and Random Forest models, while the testing partition evaluates generalization on unseen lead profiles.
2. Feature Mapping: The inputs are mapped into a model-ready feature vector containing scaled numerical metrics, one-hot encoded categorical attributes, and engineered lead-intent indicators.
3. Model Instantiation and Fitting: LogisticRegression and RandomForestClassifier are instantiated and fitted over the training matrix. The Logistic Regression model estimates interpretable linear probability relationships, while the Random Forest model learns non-linear feature interactions through an ensemble of decision trees.
4. Serialization and Persistence: Once training completes successfully, both trained models are saved as logistic_regression.pkl and random_forest.pkl. The selected champion model is serialized as

best_model.pkl, while model_metadata.pkl stores the active model name, feature list, F1-score, and ROC-AUC value for dashboard inference.

3.3 Hyperparameter Tuning

To tune and control the classification architecture, explicit model parameters are declared during training. Logistic Regression uses a stable maximum iteration setting, while Random Forest uses constrained tree parameters to balance accuracy and generalization. Key Random Forest parameters include:

- **n_estimators:** Set to define the total number of individual decision trees constructed within the forest. Increasing this value stabilizes prediction probability scores and smoothes the decision boundary, mitigating individual tree variance.
- **max_depth:** Configured to cap the maximum depth of each tree. This setting prevents individual estimators from growing indefinitely and memorizing training anomalies, forcing the ensemble to learn broader, generalized behavioral signatures instead.
- **min_samples_split & min_samples_leaf:** Set to enforce strict minimum data node thresholds required to trigger an internal split or establish a terminal leaf node. This ensures that the model does not build hyper-specific rules based on isolated outlier actions.
- random_state: Fixed across model training and data splitting to make results reproducible. Stratified splitting is used so converted and non-converted outcomes remain proportionally represented in train and test sets.

Chapter 4: Model Evaluation

4.1 Evaluation Metrics

Both candidate models are rigorously evaluated against key classification performance metrics to verify reliability and compare predictive quality. The pipeline tracks Accuracy, Precision, Recall, F1-Score, ROC-AUC, confusion matrices, and ROC curves before selecting the final model.

- **Classification Accuracy:** Measures the global proportion of totally correct predictions (both true positive conversions and true negative non-conversions) across the evaluation subset.
- **Precision Matrix:** Evaluates the model's fidelity by calculating the ratio of true positive conversions out of all leads predicted as high-intent targets. High precision ensures that sales representatives do not waste operational hours pursuing false-positive leads.
- **Sensitivity / Recall Threshold:** Calculates the ratio of true positive conversions captured out of the entire pool of actual converting clients. High sensitivity is crucial for preventing the system from letting high-yield buyers slip through uncontacted.
- **F_1-Score:** Computed as the harmonic mean of precision and recall. It serves as the primary metric for calibrating the system's performance, balancing the trade-off between lead quality and conversion capture rate.

4.2 Cross-Validation Results

To verify model generalization, the implemented pipeline uses a stratified 80-20 train-test split. The saved validation output confirms the selected champion model as Logistic Regression with F1-Score 0.7140 and ROC-AUC 0.7393 on the test set.

4.3 Confusion Matrix

The system utilizes a complete confusion matrix analysis to break down prediction errors transparently:

	Predicted Non-Conversion (0)	**Predicted Conversion (1)**
Actual Non-Conversion (0)	**True Negatives (TN):** Correctly isolated cold leads routed to background automated marketing tracks.	**False Positives (FP):** Low-intent profiles incorrectly classified as hot leads, resulting in minor outreach overhead.
Actual Conversion (1)	**False Negatives (FN):** High-intent leads missed by the algorithm, causing lost revenue opportunities if left unoptimized.	**True Positives (TP):** High-potential buyers successfully isolated and routed instantly to premium sales development loops.

4.4 Feature Importance & Explainable AI (XAI)

To interpret machine learning predictions and extract structural tree weights, Explainable AI (XAI) techniques—specifically structural feature importance analysis and SHAP (SHapley Additive exPlanations) values—are implemented directly within the data backend.

This framework analyzes individual decision nodes to extract the precise behavioral parameters and customer interaction channels contributing most significantly to lead conversion scores.

Chapter 5: Results and Findings

5.1 Model Performance Summary

The final project compares Logistic Regression and Random Forest models, then serializes the best-performing model for inference. In the current validated run, Logistic Regression is selected as the champion model with F1-Score 0.7140 and ROC-AUC 0.7393, while Random Forest remains saved for comparison and benchmarking.

The system achieves an optimal balance between precision metrics and sensitivity thresholds , ensuring that the calculated lead conversion probability indexes are highly stable. This performance directly satisfies the enterprise constraints required to deploy the model safely within live customer relationship management (CRM) architectures.

5.2 Key Findings

Granular evaluation of the predictive workspace pipeline and structural feature importance analysis reveals several critical behavioral and operational patterns:

- **Dominance of Real-Time Engagement Signatures:** Behavioral parameters such as Time Spent on Website and Pages Viewed act as primary indicators of immediate consumer intent. Extended web session durations exhibit a strong, non-linear correlation with high-yield target profiles.
- **Impact of Interaction Frequency:** Inbound touchpoints, including Form Submissions and resource Downloads, function as high-weight variables within the decision tree splits. High micro-conversion activity strongly signals an advanced position in the buying lifecycle.
- **Critical Operational Latency Windows:** The tracking system confirms that Response Time (handling latency) dramatically influences eventual closure velocity. Minimizing response time prevents high-intent target leads from going cold, protecting potential revenue pipelines from inflating acquisition costs.
- **Contextual Outreach Synergy:** Combining specific inbound channels (such as targeted email interactions or verified referrals) with strong behavioral engagement ratios creates an ideal conversion signature. This enables the automation logic to confidently categorize prospects into high-priority target bands

Chapter 6: Deployment

6.1 Tools Used

The production-ready deployment architecture integrates a high-performance stack of modern cloud technologies, development environments, and version control pipelines:

- **Streamlit Framework:** Used as the core development engine to construct the interactive web application workspace. It provides a fast, Python-native interface capable of executing live dual-mode rendering (Light/Dark workspace choices) , handling single-lead target simulations , and building multi-column analytical grid displays.
- Joblib Library: Used for model serialization and backend persistence management. The preprocessor, Logistic Regression model, Random Forest model, selected best model, and metadata are stored in the models folder so the Streamlit application can run inference instantly without re-training.
- **Plotly Engine:** Implemented as the primary data visualization system within the interactive dashboard environment. It drives high-fidelity, interactive user-interface visuals including responsive numeric probability gauge charts, outcome stream bar charts, and operational multi-variable radar profile diagnostic tools.
- **Visual Studio Code (VS Code):** Utilized as the primary Integrated Development Environment (IDE) to compose the script logic, manage file structures, debug backend processes, and embed custom CSS styling elements.
- **GitHub Repository Infrastructure:** Serves as the centralized remote source control tracking system, managing incremental version states, hosting project source files, and providing continuous integration pipelines.
- **Streamlit Cloud Deployment Platform:** Selected as the scalable production web server environment. It directly bridges with the GitHub repository branch via secure delivery webhooks to pull active source code changes and deploy the application live for global enterprise tracking and access.

6.2 Production Deployment Parameters

The end-to-end framework is securely hosted and accessible through the following verified live operational URLs:

- **Streamlit Deployed Application:** https://ai-based-sales-conversion-probability-prediction426.streamlit.app/
- **GitHub Source Code Repository:** https://github.com/akshat-042/AI-Based-Sales-Conversion-Probability-Prediction426

Chapter 7: Challenges Faced

During the design, engineering, and deployment phases of the Lead Conversion Prediction System, several technical and structural challenges were encountered and systematically resolved:

- **Handling Multi-Collinearity and High Feature Dimensionality:** Tabular customer interaction logs often exhibit heavy correlation between features—such as tracking metrics where Pages Viewed directly scales with Time Spent on Website. This multi-collinearity can destabilize traditional regression-based frameworks. The challenge was resolved by implementing a Random Forest Classifier, which selects random feature subsets at each node split to maintain stable predictive accuracy.
- **Dynamic CSS Overlay and Theme Alignment in Streamlit:** Injecting robust HTML and CSS custom sheets to force real-time theme switches (Light/Dark mode) across native Streamlit container states required meticulous styling management. Overriding standard background wrappers, sidebar boundaries, and widget text alignments required engineering a dynamic Python dictionary matrix mapped directly to st.session_state values.
- **Model Size and Deployment Constraints:** Scaling the ensemble model array with high tree estimators (n_estimators) increases memory footprints and processing time during production loading. To manage this footprint, the training pipeline uses precise tree-depth constraints (max_depth) alongside efficient serialization libraries (Joblib and Pickle) to package the model binary file for instant cloud deployment without execution latency.
- Feature Synchronization Between Pipeline and Dashboard Frontend: The machine learning algorithm expects the same scaled, encoded, and engineered feature layout used during training. Ensuring that Streamlit inputs and CSV uploads map accurately to the saved preprocessor required dedicated transformation logic to avoid shape mismatch errors during live inference.

Chapter 8: Future Enhancements

To scale the Lead Conversion Prediction System into a more robust, enterprise-grade cognitive platform, several critical architectural and functional advancements are planned for future development cycles:

- Transition to Advanced Boosting Architectures: Future iterations will evaluate models beyond the current Logistic Regression and Random Forest baseline, including XGBoost, LightGBM, and CatBoost. These models can improve predictive power on larger CRM datasets.
- **Integration of Real-Time Streaming Data Pipelines:** The data ingestion engine can be extended to support event-driven, real-time message streaming protocols (such as Apache Kafka or AWS Kinesis). This will allow the machine learning pipeline to process live user actions instantly as they happen on enterprise websites, rather than relying on batch CRM updates.
- **Automated Hyperparameter Tuning and MLOps Workflows:** Implementing automated machine learning pipelines (AutoML) alongside hyperparameter optimization tools (like Optuna) will enable automated tuning loops. Coupling this with MLOps tracking platforms (such as MLflow) will make it easier to manage model versions, monitor performance drift, and handle seamless retraining schedules.
- **Advanced Deep Learning and NLP Embeddings:** For richer profile analysis, the feature engineering pipeline can incorporate Deep Learning architectures. Integrating Natural Language Processing (NLP) models (like BERT or custom transformers) will allow the system to analyze unstructured text data, such as live customer support chat transcripts or email query logs, turning raw text into predictive intent signals.
- **Full-Scale CRM API Bi-Directional Integrations:** Expanding the current integration layout to support seamless, bi-directional API webhooks with major enterprise CRM ecosystems (such as Salesforce, HubSpot, or Microsoft Dynamics) will allow the model to automatically sync its prediction outputs. This ensures high-intent alerts trigger automated workflows directly inside sales teams' primary workspaces.

Chapter 9: Conclusion

The development and deployment of the Lead Conversion Prediction System successfully demonstrates how integrating advanced machine learning pipelines with premium interactive workspace dashboards can optimize enterprise sales environments. By replacing static pipeline tracking and subjective manual logging with a highly optimized, automated Random Forest Classification Engine, the system effectively captures hidden, non-linear relationships across diverse customer behaviors.

The technical architecture seamlessly ingests multi-dimensional consumer interaction arrays, applies structured preprocessing rules, and generates real-time, high-fidelity conversion probability indexes. This predictive power, combined with tree-based Explainable AI (XAI) feature relevance modeling, ensures that revenue teams can interpret exact behavioral intent metrics without encountering "black box" analytical limitations.

Furthermore, the deployment of the secure, dual-mode web platform on Streamlit Cloud enables global accessibility and direct integration with continuous source-control pipelines. Ultimately, this project delivers a robust data-driven decision support framework that systematically minimizes customer acquisition costs (CAC), enhances sales handling velocity, and maximizes long-term pipeline reliability—bridging the gap between predictive artificial intelligence and strategic corporate execution.

Chapter 10: References

The development, algorithm selection, and architectural design of the Lead Conversion Prediction System were informed by established academic research papers, formal library documentation, and industry-standard machine learning texts:

- **[1] Bank Marketing and Lead Conversion Analysis:** Moro, S., Cortez, P., and Rita, P., "A Divide-and-Conquer Strategy Using Feature Relevance and Expert Knowledge for Enhancing a Data Mining Approach to Bank Telemarketing," *Expert Systems*, vol. 35, no. 3, 2018. DOI: 10.1111/exsy.12253.
- **[2] Machine Learning Core Infrastructure (Scikit-learn):** Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- **[3] Core Algorithmic Foundation (Random Forests):** Breiman, L., "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
- **[4] Random Forest Classification Engine Technical Reference:** Scikit-learn Official Documentation, *RandomForestClassifier Reference Guide*.
- **[5] Explainable AI (XAI) and Model Interpretation:** Lundberg, S. M. and Lee, S.-I., "A Unified Approach to Interpreting Model Predictions," *Advances in Neural Information Processing Systems*, 2017. (SHAP Framework Paper).
- **[6] Structured Tabular Data Engineering Tools:** Pandas Development Team, *Pandas Software Documentation for Data Manipulation*.
- **[7] High-Performance Matrix and Numerical Computations:** Harris, C. R. et al., "Array Programming with NumPy," *Nature*, vol. 585, pp. 357-362, 2020. DOI: 10.1038/s41586-020-2649-2.
- **[8] High-Fidelity Visual Analytics and Interactive Charts:** Plotly Python Graphing Library Open-Source Documentation.
- **[9] Object Serialization and State Persistence Pipelines:** Joblib Development Group, *Joblib Toolset Utilities Documentation*.
- **[10] Interactive UI Workspace Dashboard Deployment:** Streamlit Framework Open-Source API Reference Documentation.
- **[11] Applied Predictive Modeling and Machine Learning Workflows:** Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed., O'Reilly Media, 2019.
- **[12] Deep Learning Foundations and Advanced Pattern Networks:** Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*, MIT Press, 2016.